Silvia Travieso

DESIGN TECHNOLOGIST

I build products and I design systems.

and If I can't find the right tool for the job, I'll build that too.

Product Designer × Design Systems Specialist

Meet Me!

Born and raised in the Canary Islands, now living between there and Barcelona.
Heavily shaped by British culture and humour.
I have a lot of plants in my apartment and bake way too much focaccia weekly.

FOUNDATIONS

`Architect → Visual Designer → Product Designer → Design Technologist`
I trained and worked as an architect before moving into design, and that training has stayed with me: problem solving, thinking on how decisions stack on top of each other, what happens when things fail and how to prevent it.

MY KIND OF VIBE

`Design systems aren't a trend for me. They're just how I think.`

`SaaS` `B2B` `data-heavy` `CRM` `internal tools` `audits` `token architecture` `automations`

EXPERIENCE

Senior Design System Strategist | Nateevo (VASS)
2026 – present

Client: CaixaBank (Fintech). Bridging design-dev on a multi-brand design system and building design ops processes and tools.

Product Designer | Tomato Energy
2025

B2C interfaces for onboarding energy customer via a microsite with real-time status, magic-link access and a self-diagnosis tool, replacing reactive email updates and reducing anxious support calls.

Product Designer (promoted to Design Systems Lead) | Senapt
2024 – 2025

B2B SaaS product for marketing teams on energy companies, leading end-to-end design across multiple products from discovery through MVP.

linkedin.com/in/silvia-travieso-gonzalez/

https://github.com/SilvT

Design Ops / Plugin

Library Freeze System

International Bank, via Nateevo • 2026

House-cleaning an 8-designer working at the same time across a 4-library design system.

Devised a freeze process to align design and development on a shared sprint clock, then built a Figma plugin to log, compile and diff changelogs across the libraries. Full adoption within two months, zero bugs across four freeze cycles, now self-published on Figma Community.

→ Read Case Study

INFORMATION

Project Type
Design Ops / Plugin

Role
Design Technologist

Team
Consumed by 8 designers
Tooling built solo

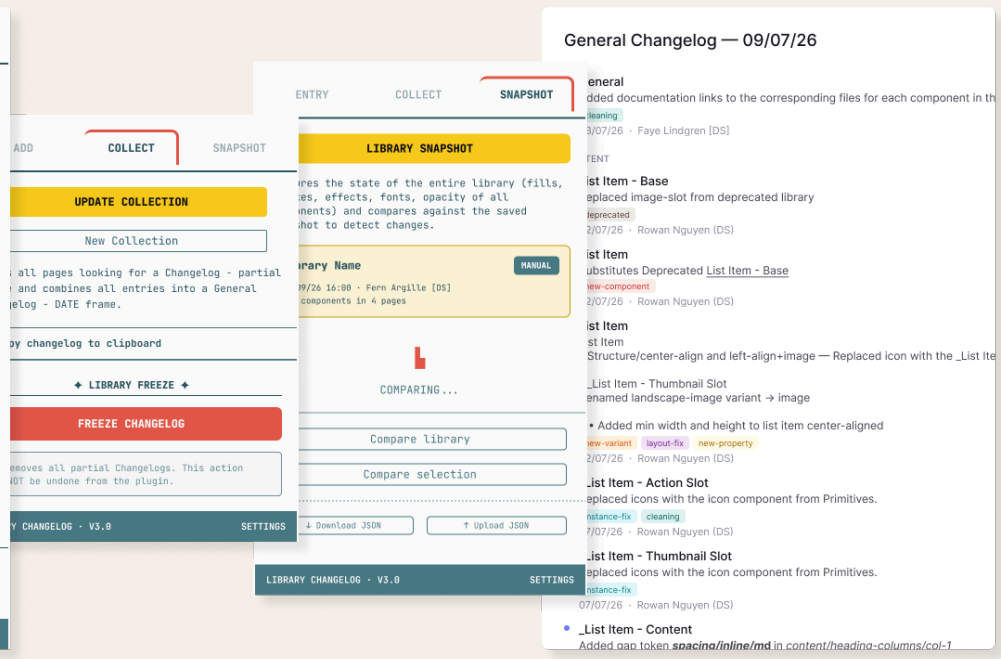
Stack
Figma Plugin and REST API, LLM integration

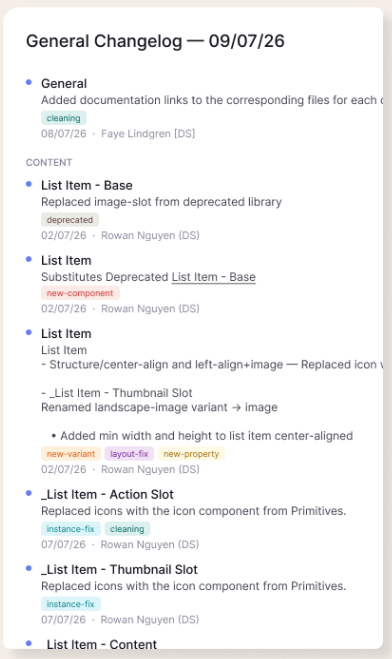
4
Freeze cycles, zero bugs

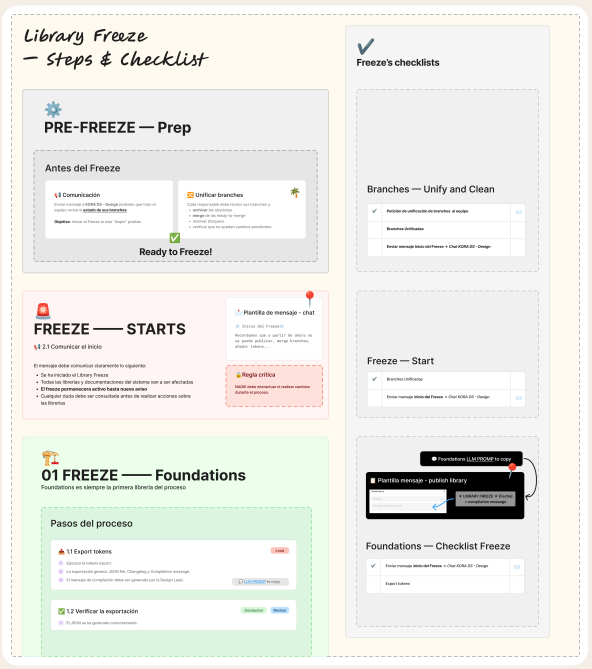
8
Designers onboarded

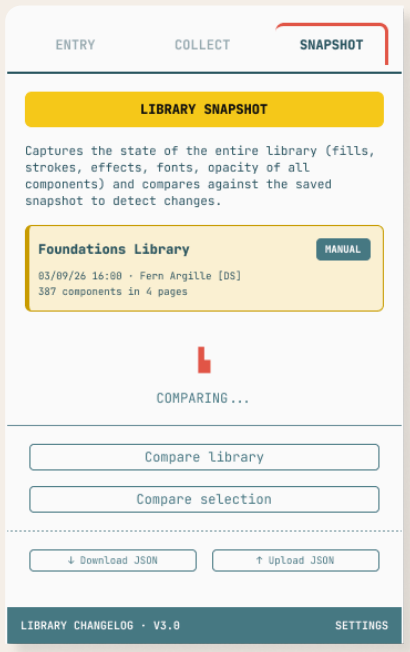
~3K
Components tracked













Design Ops
Figma Plugin Dev
Generative Plugins
Design Systems
LLM Integration

Product

Marketing Management

Senapt LTD • 2024 – 2025

From Mid UI to Product Design Lead: Building Enterprise Software in a Startup

Desktop App, built from scratch to unify fragmented marketing operations across 4 departments, replacing manual Excel workflows with real-time data visibility. Custom-made for an electrical company client and their particular flows. A lean team of one back-end engineer, one front-end engineer and me as the solo designer achieved an MVP in 1 year. This led me to quickly grow into product co-owner and product designer, and later Lead of Design Systems with 7 designers under my supervision.

INFORMATION

Project Type
B2B · Desktop App

Role
Mid UI Designer → Product Co-owner

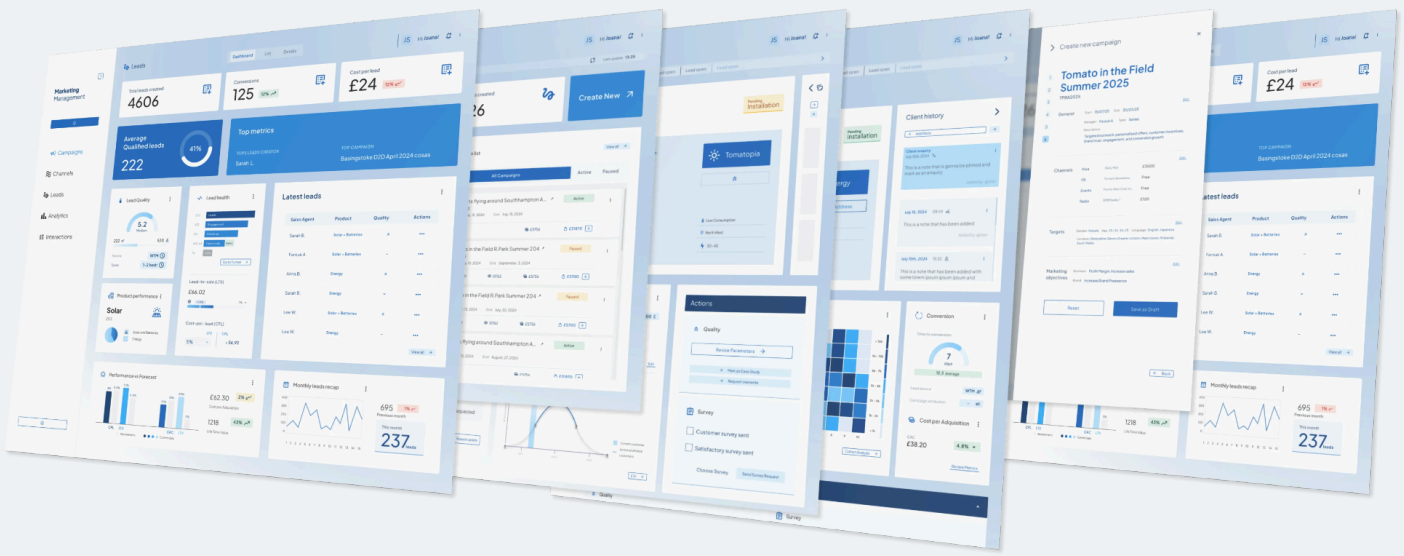
Team
1 Backend Lead + 1 Frontend + Mid UI (me)

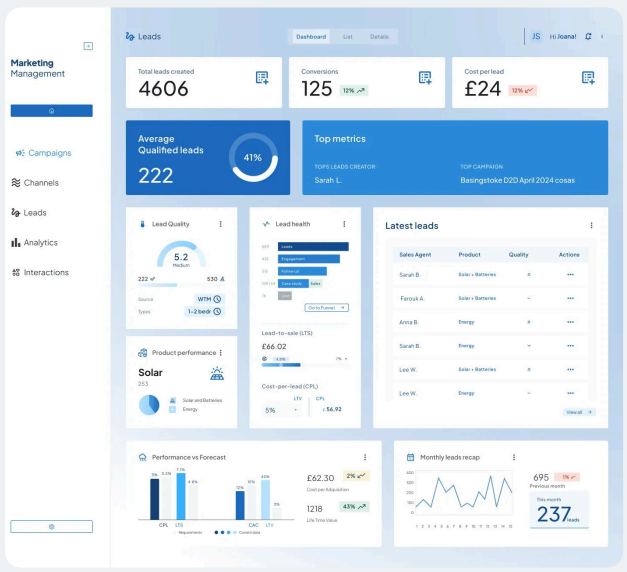
Stack
Java FX

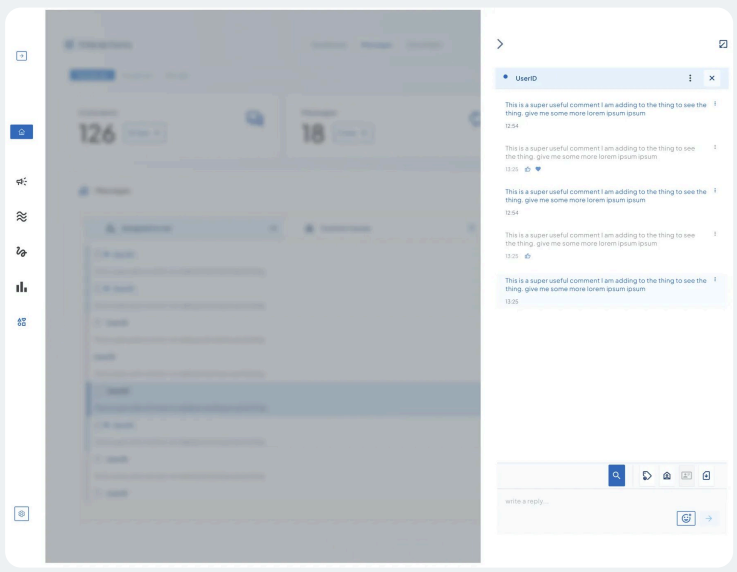
50%
Customer growth

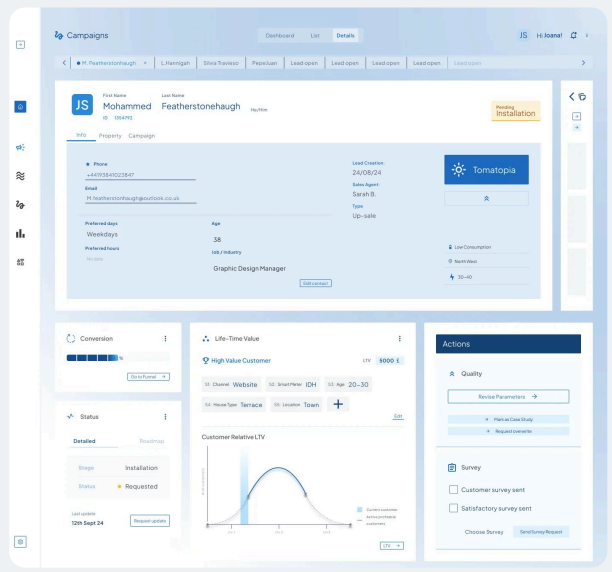
4 → 1
Unified Platform

75%
Reduced Manual work









Product Design · UI/UX · Design Systems · B2B · 0→1 · Co-ownership

Design System

Design System from Scratch

Senapt LTD • 2024 – 2025

No file naming conventions, no libraries, no precedents and no one at the company who yet believed in design systems.

Built a production-ready design system from scratch as a solo mid-level designer in an engineer-first startup. Worked around an unusual stack, a non-web environment, a fully async front-end relationship and CEO resistance. The result came with a 4-level token architecture, an experimental, prototype-like use of Variable modes, and a personalised Figma-to-code pipeline via Token Studio + GitHub Actions, auto-triggered in cascade. This system became the go-to example, triggering a promotion and an educational and onboarding process for the other 7 designers in the department.

→ Read Case Study

INFORMATION

Project Type
Library / Repo

Role
Mid UI Designer → Design Systems Lead

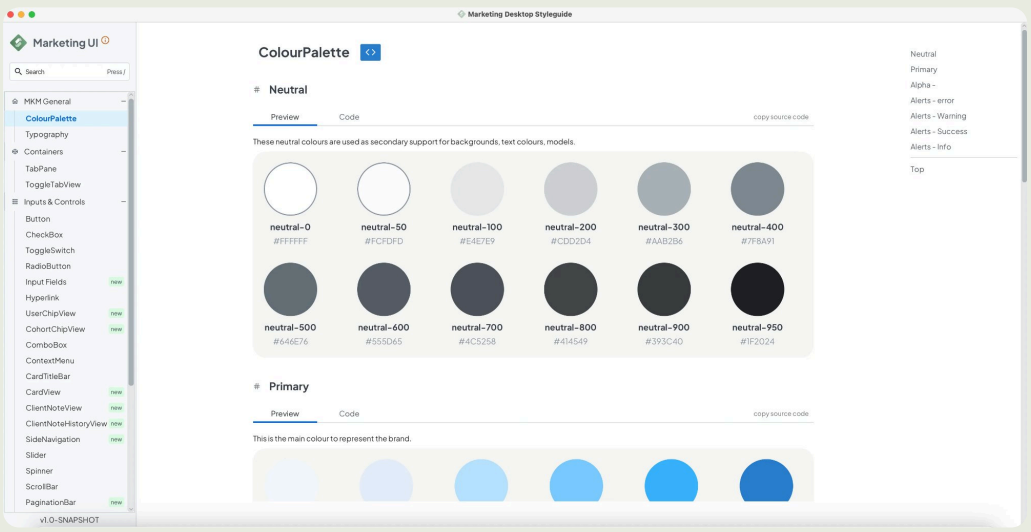
Team
Solo → scaled with 8 designers

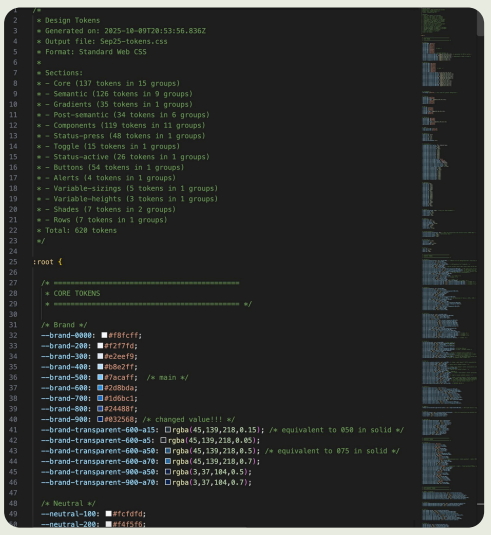
Stack
Figma Variables, JavaFX, GitHub Actions

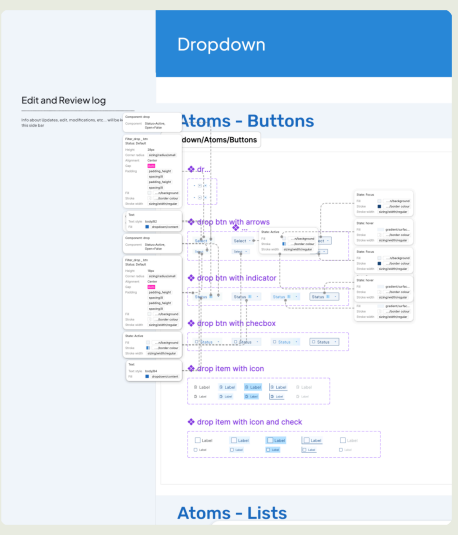
90%
QA reduction

864
Production tokens

7
Projects adopted







Design Systems | Token Architecture | JavaFX | GitHub Actions | Greenfield 0 → 1

Product

Energy Switching Tracker

Tomato Energy • 2025

Helping customers understand their energy journey

A well-known pain point for users in the UK's energy market, and an unclear journey. After hunting down backend engineers to map a process no one fully understood, I built a customer-facing tracker that replaced reactive 'we'll email you' communication with proactive self-service, saving Customer Service from a lot of anxious phone calls. The UI is now connected to the backend, so it always shows the most up-to-date information, ready to consume visually. Clear steps, hints of the whole journey and a progress bar, while keeping the quirky and fun brand elements of the energy company.

INFORMATION

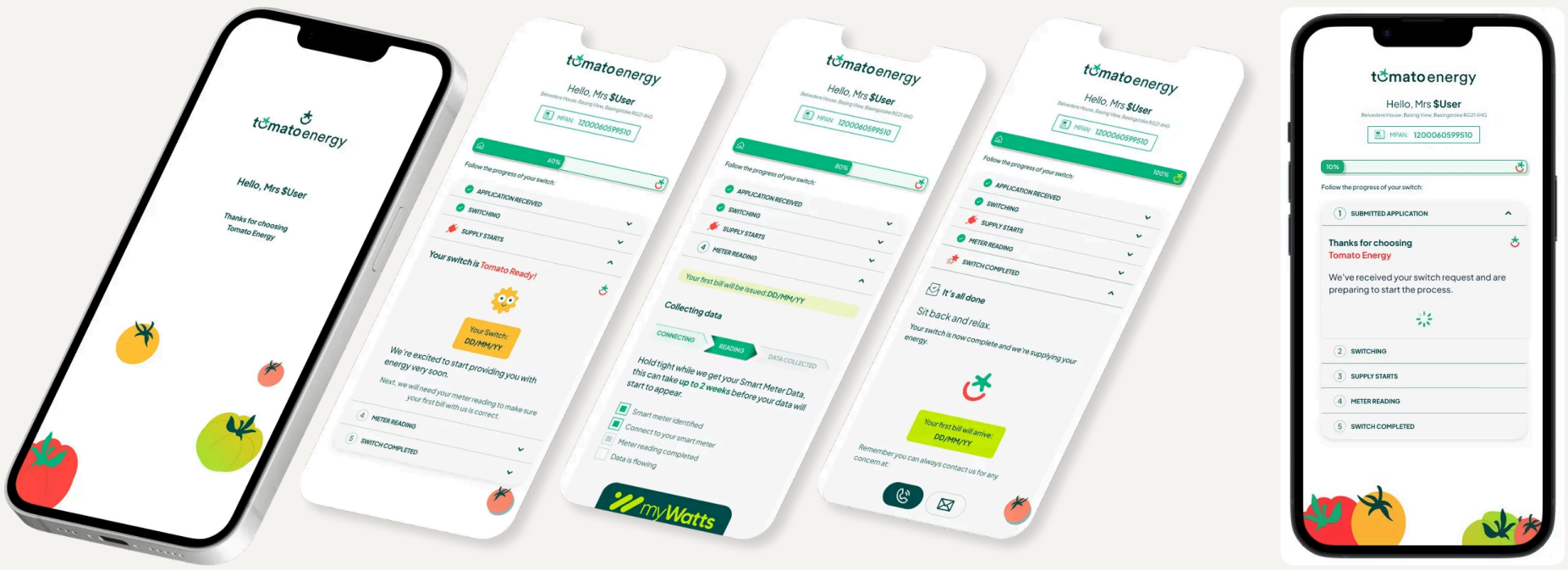
Project Type
B2C · Web

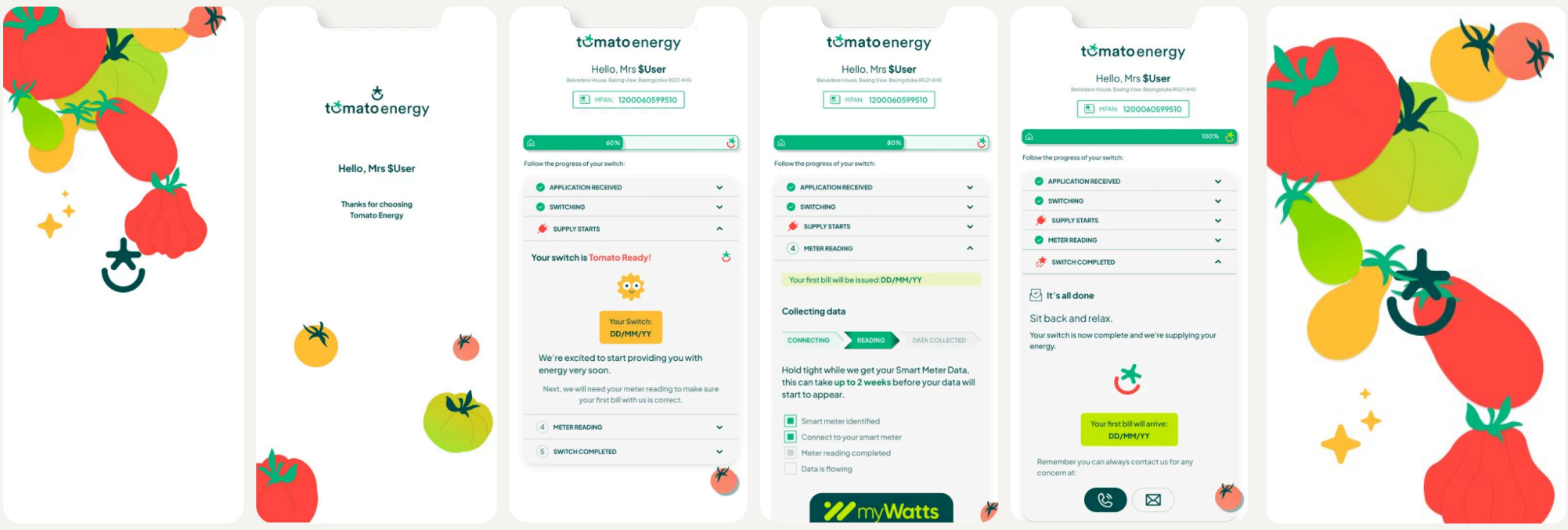
Team
Solo Designer

Stack
Nuxt - Mobile First

Role
UI/UX Designer

Real time	1	Customer
Updated data	Magic link	Self-diagnosis tool





Product Design · UI/UX · B2C · Mobile-First · Data Visualization

TOOLS

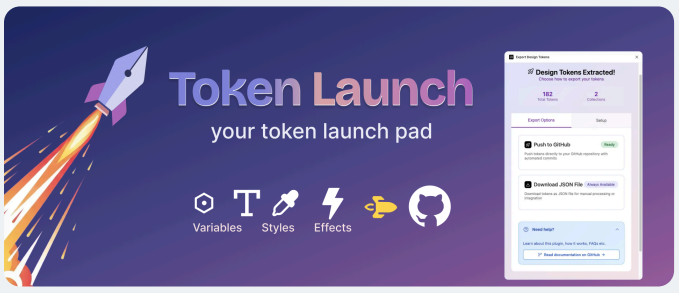
Custom tools, self-developed.

Small tools built to solve specific design-system frictions that have expanded beyond their original scope and are now widely available for consumption on Figma Community.

✓ Published

Token Launch

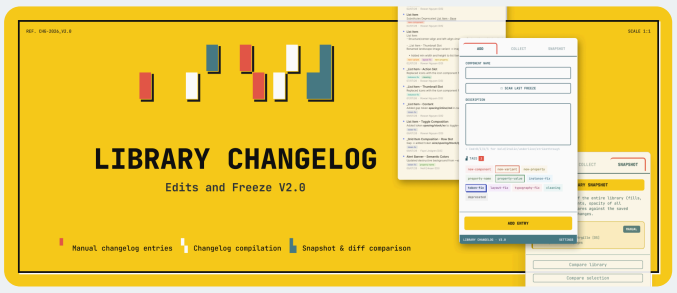
Identified a critical gap in design token workflows across 3 product teams and built a lightweight Figma Variables → GitHub sync.



✓ Published

Library Changelog

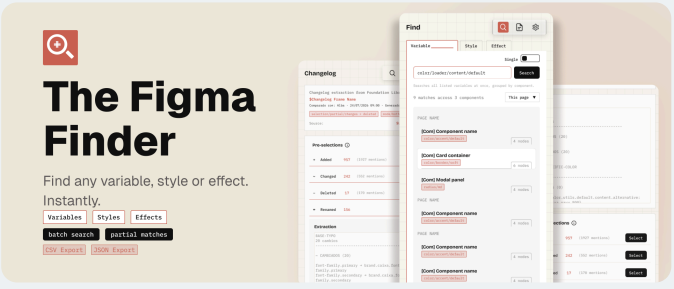
A Figma plugin that keeps library changelogs consistent across designers, from first entry to release diff.



Coming soon

The Figma Finder

Find any variable, style or effect instantly, with batch search, partial matches, and CSV/JSON export.





Library Freeze System

A process and a plugin: a full system to provide stability to a 4-library design system with multiple designers.

01

INFORMATION

Project Type
Design Ops / Plugin

Role
Design Technologist

Company
International Bank (via Nateevo)

Team
Consumed by 8 designers
Tooling built solo

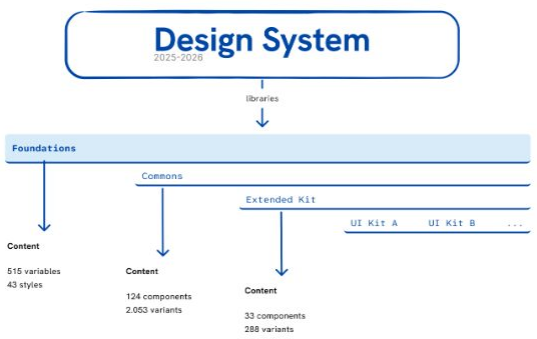
Stack
Figma Plugin and REST API, LLM integration

Timeline
Mar–Sep 2026

Eight designers, four library files, and no record of what changed between one sprint and the next.

I joined this multi-brand design system about six months into the project, brought in as Senior Design System Strategist via Nateevo. The foundation was already in place (variables, a commons/brand split, half the first component batch built), yet with up to 8 designers working on the libraries at the same time there was no process for tracking ownership of changes across sprints. Developers needed to know what had shifted before pulling an update and who to ask for clarifications; designers needed to tell each other what they'd touched. In practice, that happened in Teams messages and meetings, and got lost the moment the library grew past a certain size. Inspired by the development team's code freezes per sprint, I built the process and the tooling to fix that: a Figma freeze cycle and a generative Figma plugin that logs, compiles and summarises changes as they happen.

Context



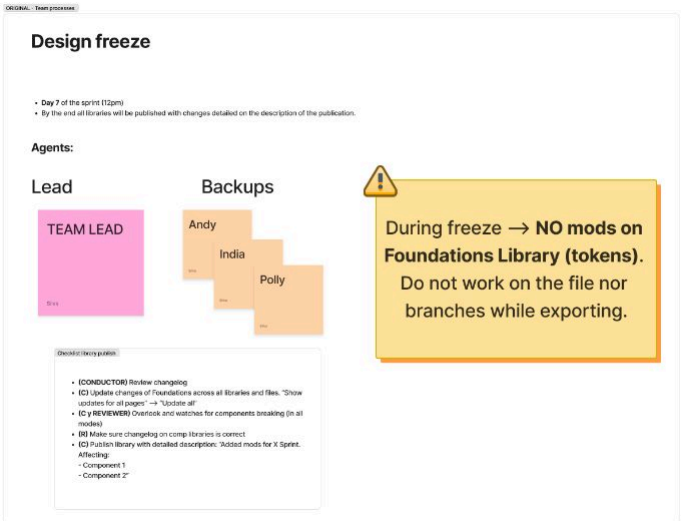
--context-diagram- the 4-library structure holding around 3K components

The starting point: 3000 components but no tracing.

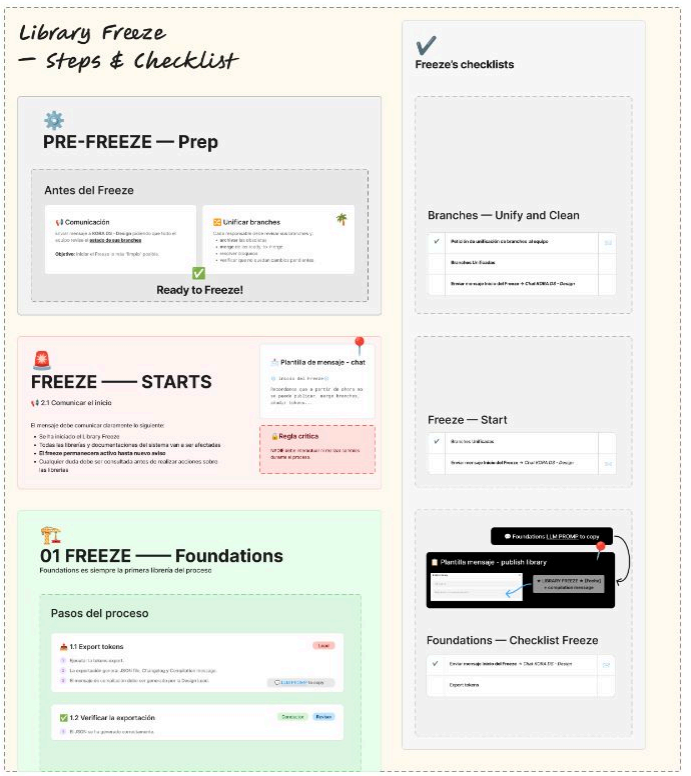
Four levels, variables shared across brands, a commons layer, an extended commons for retail brands, and (at that point) two brand-specific UI kits: Brand A, web-first, and Brand B, app-first. Together holding around 3000 components counting variants individually and 8 designers working across them daily, pulled in from several different consulting companies, most with no prior design systems experience and no clear hierarchy despite a nominal lead. Separately, the dev team pulled from these libraries on 2-week sprints that mirrored the design cycle. The problem wasn't the size of the libraries. It was that, between each sprint, there was no record of what had changed or who changed it, and the bigger the libraries grew, the worse that got.

Before building anything we needed to set a process, not just a tool: a library freeze aligned to the dev team's sprint. At the end of each sprint, design freezes the library at a known state; development pulls from that save point. It gave both teams a shared clock; everyone knew which version they were aligned to, and a changed library stopped being something developers discovered mid-sprint. The plugin came after the process, not instead of it, to make that freeze cheap to run and easy to trust, rather than another manual step someone would eventually skip.

Security policies within the Client's Figma environment required all plugins to be requested via a formal acceptance process. Any new plugin needed admin approval before the test cycle, and that timeline ran longer than it would've taken me to just build a custom one myself.
So I did.
Not to bypass process for its own sake, but because the team needed the tool before the request cycle would have delivered it.



--first-draft- a plain bullet-list doc with day/time rules, one lead, one backup.



--doc- What's running now: a phased operational board, kept open during every freeze.

01 **Pre-Freeze**

Branches unified, team alerted

02 **Foundations**

Day 7, from noon

Tokens exported, Foundations published

03 **Commons / Extended**

Same day

Changelogs compiled, libraries published

04 **Documentation**

Independent, can extend to next day

Docs updated to match the freeze

Draft, Test, Validate.

It started as a page of rules. Freeze on day 7, starts at noon, etc. Six rounds in, the process became clearer with each one, landing on the current structure: 5 defined stages, a streamlined process, clear ownership, roles and outcome criteria. Documentation was updated and made ready for future onboarding to the team, separating each phase with its own checklist, support templates and prompts.

Clear rules and red lines were set for the team: Foundations must be left untouched once a freeze starts; anything that comes up gets queued for the next sprint instead.
With a larger design systems team to draw on, the role of conductor of the freeze rotates, with this sprint's conductor becoming next sprint's reviewer, so the process doesn't depend on one person remembering how it works.

Underneath it all: the need for a tool.

With a system now in place, a hidden friction point came into clearer view. We were relying on what was pending to be published on each library as the only, and very unreliable, source of changes across the libraries. I devised a POC changelog using a Notes component from our toolkit, to be used and left by designers next to the components touched, iterated or created, so they could all be collected by the time the freeze came around.

This became the foundation and defined the PRD of the Changelog Plugin.

The friction was deliberately kept as low as possible. We were asking designers to keep in mind, and remember, that they had to log the changes right after they were done.
Key decisions to achieve this:

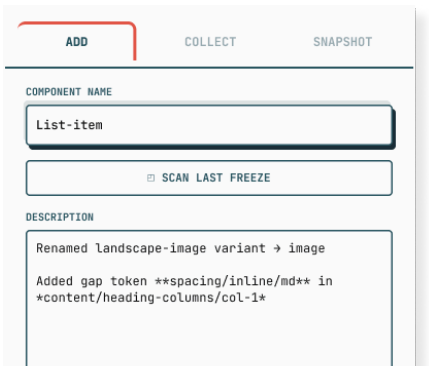
- **Figma Generative plugin instead of traditional plugin:** an unexpected benefit of Figma's own generative plugins is that they are bound to the files they are run in AND they appear on the right action bar. No need to remember to launch a plugin: it's always at hand and ready to use.
- **Auto detect and auto fill:** component's name is filled by simply selecting the component on the canvas. User name and timestamp are autogenerated, removing the ownership traceability problems.
- **Daily use first and upfront:** the plugin opens ready to log a partial entry. No navigation inside needed. Freeze and Snapshot functions are a couple clicks away, with a double verification built in to avoid unwanted overwrites or untimely compilations.

The Partial Changelog entry tab in particular needed almost no explanation; the lower the friction to log a change in the moment, the more likely anyone actually does it.

Outcomes

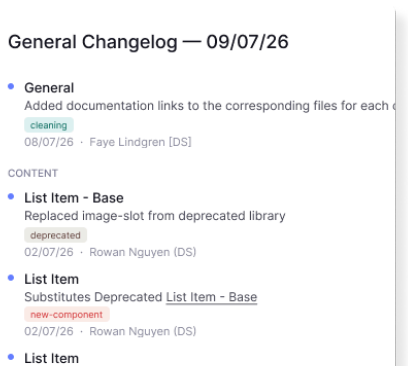
Full adoption of the process and plugin within the first two months.
Freeze documentation and steps ready for onboarding.
Across 4 successful full freeze runs, the plugin has performed with no bugs detected and zero pushback. The plugin is now self-published on Figma Community.

The Plugin: Functionalities



Add entry

Logged by designers as they work. Click a component, the plugin pre-fills name, user, timestamp; you add the note. Colour-coded by change type. No context switching, nothing to remember later.



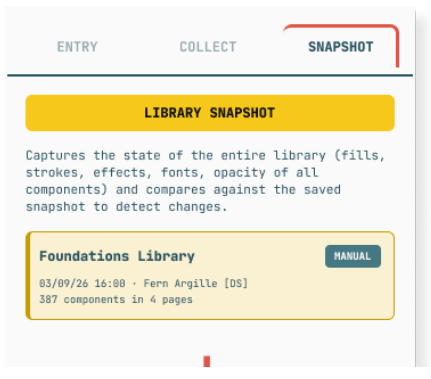
General and Partial Changelog

Collected at the end of each sprint on its own dedicated page.



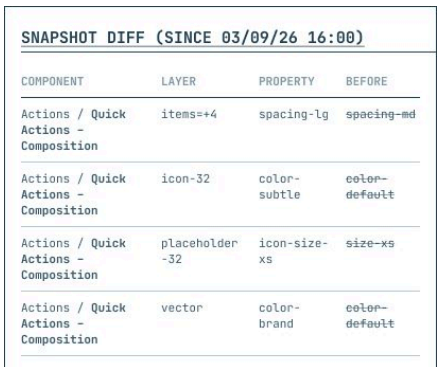
Freeze

Run once per sprint, by the lead. Compiles every scattered partial entry into one changelog, creates a named save point in Figma's version history, and feeds the compiled log to an LLM to generate a plain-language publish summary for developers.



Snapshot

A JSON snapshot of every variable value at freeze time. Each new freeze diffs against the last, auto-detecting token changes instead of anyone hunting for them.



Diff

View the list of differences on a downloadable file.

Learnings

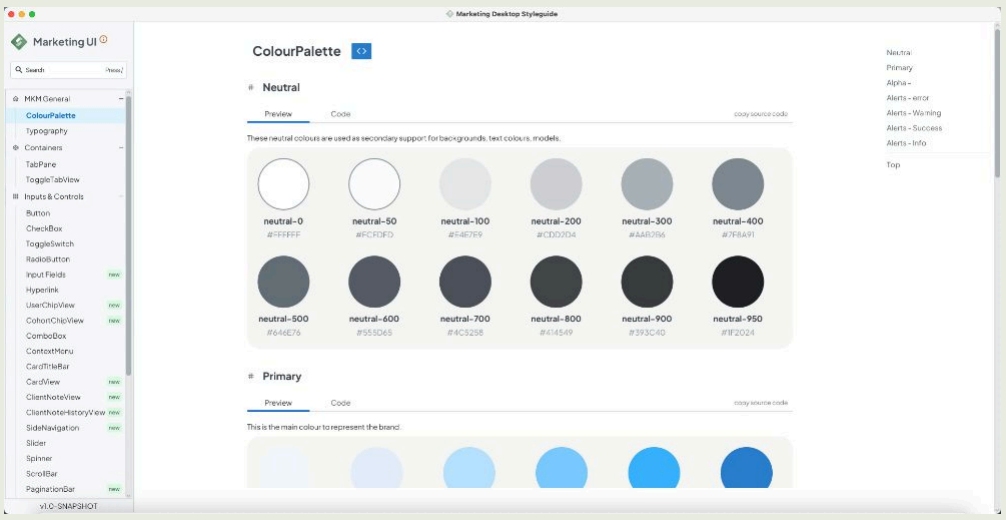
Honestly, not much went wrong here, and that's worth saying plainly rather than inventing a struggle to sound reflective. The team wanted structure, the process held up sprint after sprint, the plugin didn't need a single bug fix. That's rare enough that I don't want to manufacture a lesson where there wasn't a real one.

If I'm pushing for something genuine: the system was never tested against a third brand joining the commons library, so we proved it works at the scale I actually ran it at, not necessarily beyond that. And the plugin fixes the friction of logging a change, but not the moment someone simply moves on without logging it at all; that's a timing gap, not a UX one.

Now that it's available on Figma Community, being used outside the one environment it was built around, I'm looking forward to seeing whatever bugs or edge cases that may surface in various environments and scales.

Test it yourself!

Design System from Scratch



Design infrastructure built solo, with no precedent, inside an engineer-first startup.

02

INFORMATION

Project Type
System

Role
Mid UI Designer → Design Systems Lead

Company
Senapt LTD (Startup)

Team
Solo → scaled with 8 designers

Stack
Figma Variables, JavaFX, GitHub Actions

Timeline
~1 year 3 months to MVP

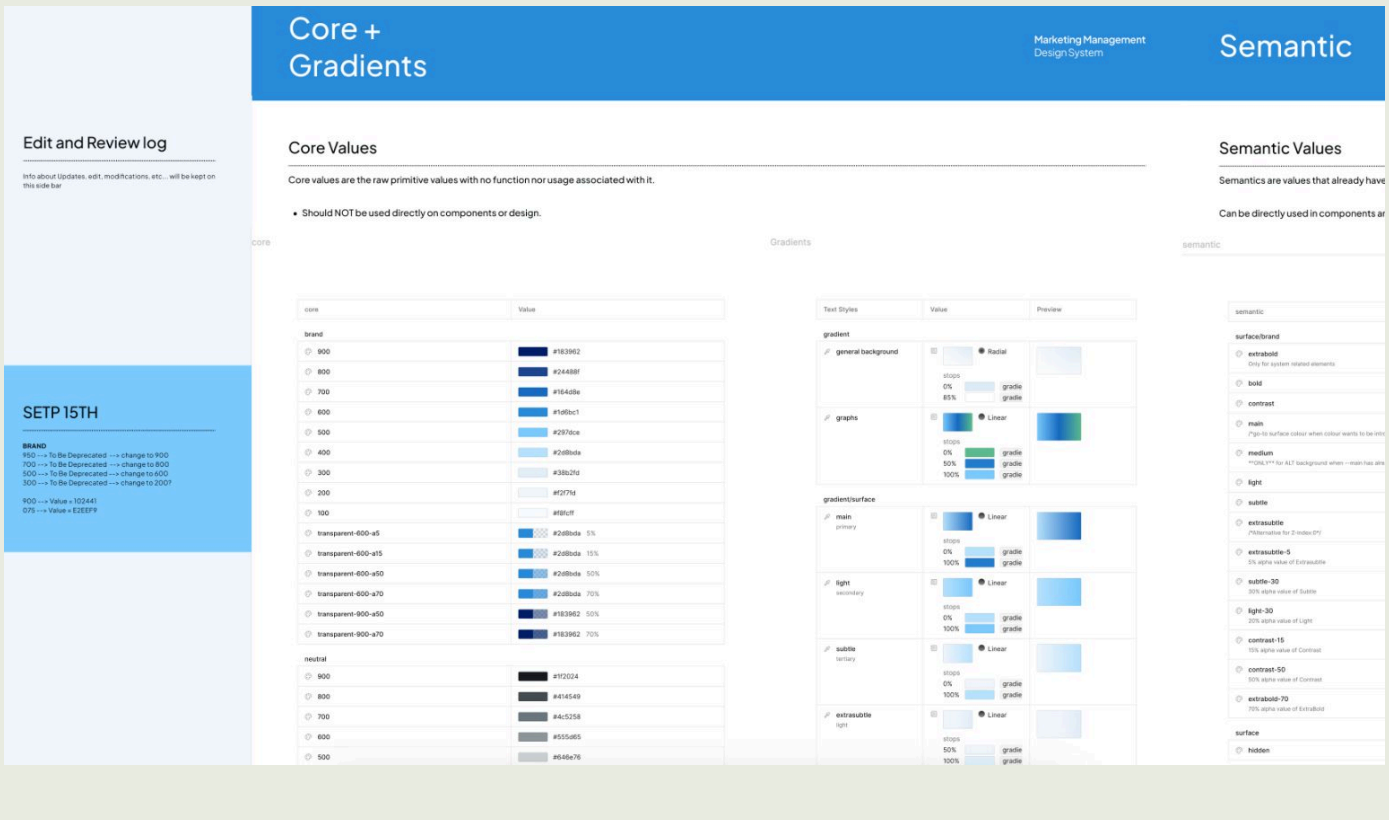
No file naming conventions, no shared libraries, no design structure, and no one at the company who yet believed in design systems.

I joined Senapt as its second full designer, and was handed their fourth product line Marketing Management desktop app: a campaigns, leads and channels tracking dashboard. Designed solo opposite a backend developer and a frontend developer seven time zones ahead in Japan, who read written English but wasn't a fluent speaker. Handing off ambiguously meant every misunderstanding became a huge delay. Across the other product lines, designs were taking four or five review rounds after hand-off, and developers sometimes shipped straight to a live product without design sign-off.

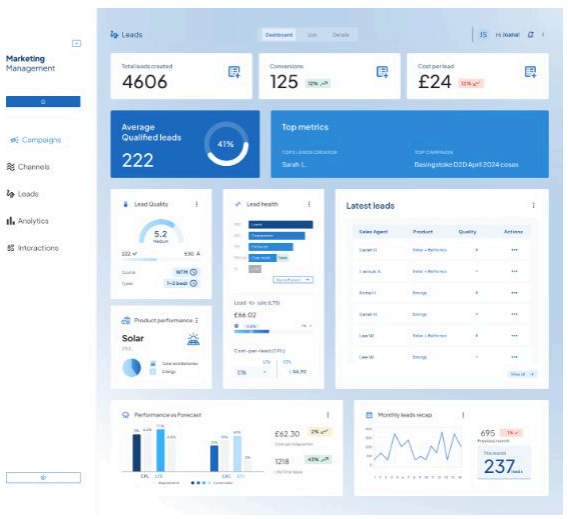
My goal was to build this product component-first and systematised from day one, to close that gap before it even opened.

This design system was product specific, yet aimed at the whole company. My first pitch to the CEO of a design system approach overall was turned down outright, yet achieved a tentative yes from the VP of Frontend (though from a POV of bypassing designers in the future). I didn't love the starting point, so instead of fully backing down, I used this app as a Proof of Concept rather than waiting for an approval that the stakeholders weren't ready to give.

“Products are working. Nobody's complained. This isn't a priority.”
– CEO (Feb 2024)



Context: The First Drafts



`--look-and-feel`–screen that got approved

No buy-in? Lead by example

Three existing product lines, each built by freelancers with no systematic mindset behind them, had accumulated their own disconnected UX, playgrounds and UI files: no naming conventions, no shared libraries, hand-offs kept manually in sync with live product by whoever had time. After my pitch was turned down and I decided to use this product as a proof of concept anyway, I got to work laying strong foundations and building as stable a structure as I could.

Scaffolded in Phases, building on the go

I was juggling key product feature decisions, research, UX flows, UI, hand-offs, QA, reviews... I was the solo designer in this.

The system was never going to land in one pass, nor be sturdy from the beginning, so it didn't try to. Right after getting approval on the look and feel, I launched into Phase 1: prove the core tokens, type, and a first handful of components, enough to build the dashboard's first complete flow. Phase 2 expanded it: semantic tokens, gradients and elevations, a wider component set that grew over time out of strict necessity. Phase 3 pushed into solidifying, looking back and reinforcing where it had become wobbly.

Delivery and hand-offs also followed a set order: components first, screen sections, full screens, then flows. Every time, changes cascaded predictably because of the architecture underneath.

The Bumps on the Road(map)

01 **A Weak Semantic Layer**

Months 1–5

Thin semantic foundations, built with no dedicated time and no existing UI to learn from.

02 **Semantic, Rebuilt From Real Usage**

Month 5–6

Caught before it hardened into a habit, and rebuilt from what the screens actually needed.

03 **Modes Experiments**

Month 5–8

Merging a traditional and prototype approach to modes on Figma variables.

04 **Fourth Level**

Month 10–12

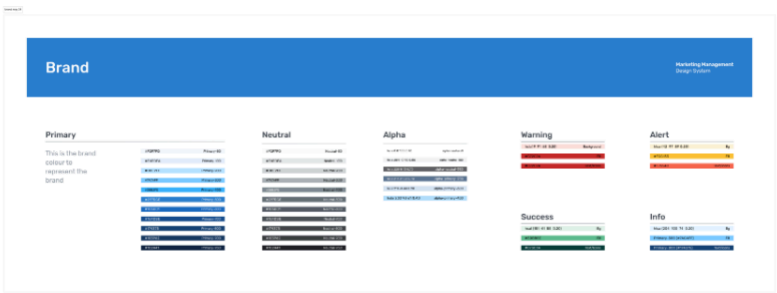
A custom fit: closing a gap with no industry reference, solved by adding a custom in-between layer.

The semantic layer was wrong twice, in two different ways.

✦ As my first time setting up a design system from zero, newbie errors were to be expected. The first iteration of semantics was thin, loosely drawn from an outside reference I no longer remember, and token creation quickly skipped past it, with component tokens piling up connected straight to primitives. With no dedicated time to build the system properly, about five to six months in, the pattern was obvious and quickly growing unmanageable. I gave my frontend developer a heads-up that a large structural change was coming, then scrapped the original semantic layer and rebuilt it from what the screens were actually using. Now that a full UX flow had been completed, I had a much clearer view of how I, myself, was using colours and values, so extracting the pattern felt more natural.

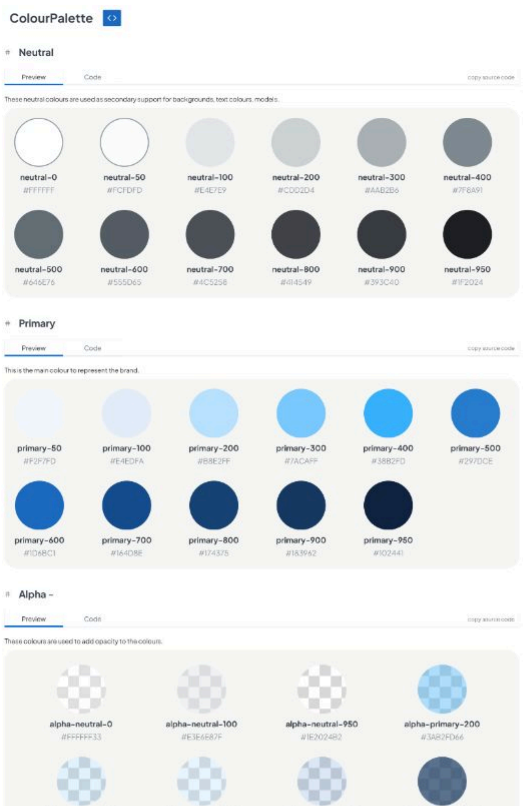
✦ That rebuilt semantics held for a while, but around month ten to twelve, I noticed something else: patterns repeated across many components, like text or cards having the same hover/focus behaviour everywhere regardless of whether they were nested, colour usage on elevations, disabled statuses on fills/texts/borders/backgrounds... which weren't semantic in the usual sense but weren't component-specific either. Bunching them into the current semantic layer felt conceptually weak, yet a full rebuild didn't make sense. I looked for examples or precedent in other systems and found none, so I outlined a fourth level, sitting between semantic and component, and called it post-semantic. The main goal of this layer: to hold the cross-component patterns I was coming across.

✦ From the get-go, dark mode was off the table, which freed up a more experimental use of Figma Variables modes. Closer to prototyping than traditional system-building, smaller collections scoped only to component-level tokens were created to match recurring state patterns: default/active/focus/hover/disabled, default/pressed/focus/hover/disabled, on/off, xs/s/m/l/xl, primary/secondary/tertiary. This choice kept the system contained in terms of number of variables and sped up the breakdown for delivery of full UX flows.



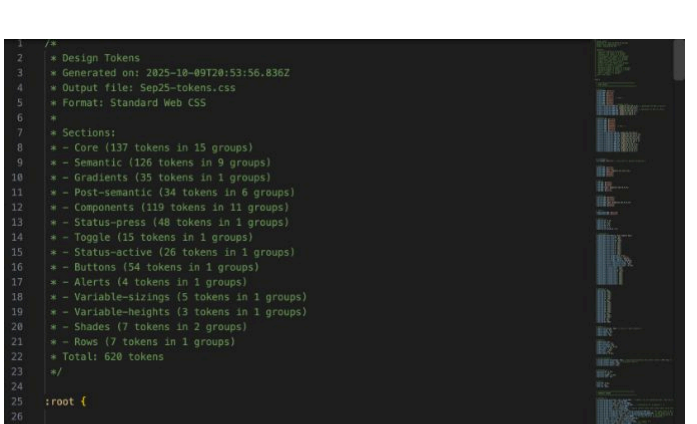
--iteration-1- A rough first layer of tokens was laid out to have something to work on top of.

The System



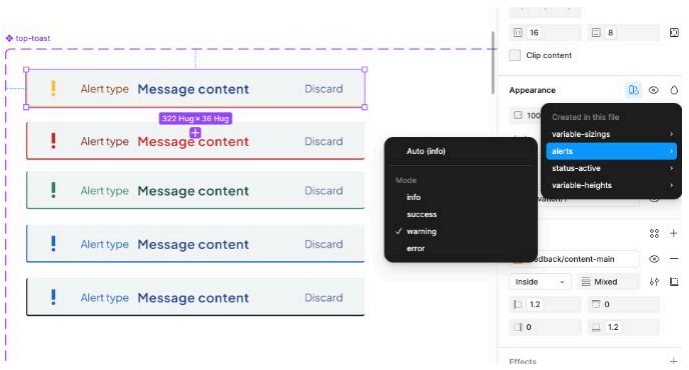
Foundation Architecture

System set up in a traditional 3-tier structure with a weak semantic layer that is strengthened further down the line.



A Fourth Level

Post-semantic sits between semantic and component: for patterns that repeated across components (text states, backgrounds, elevations, disabled elements...) without being semantic in the traditional sense. The name is exactly what it sounds like: what comes after semantic.



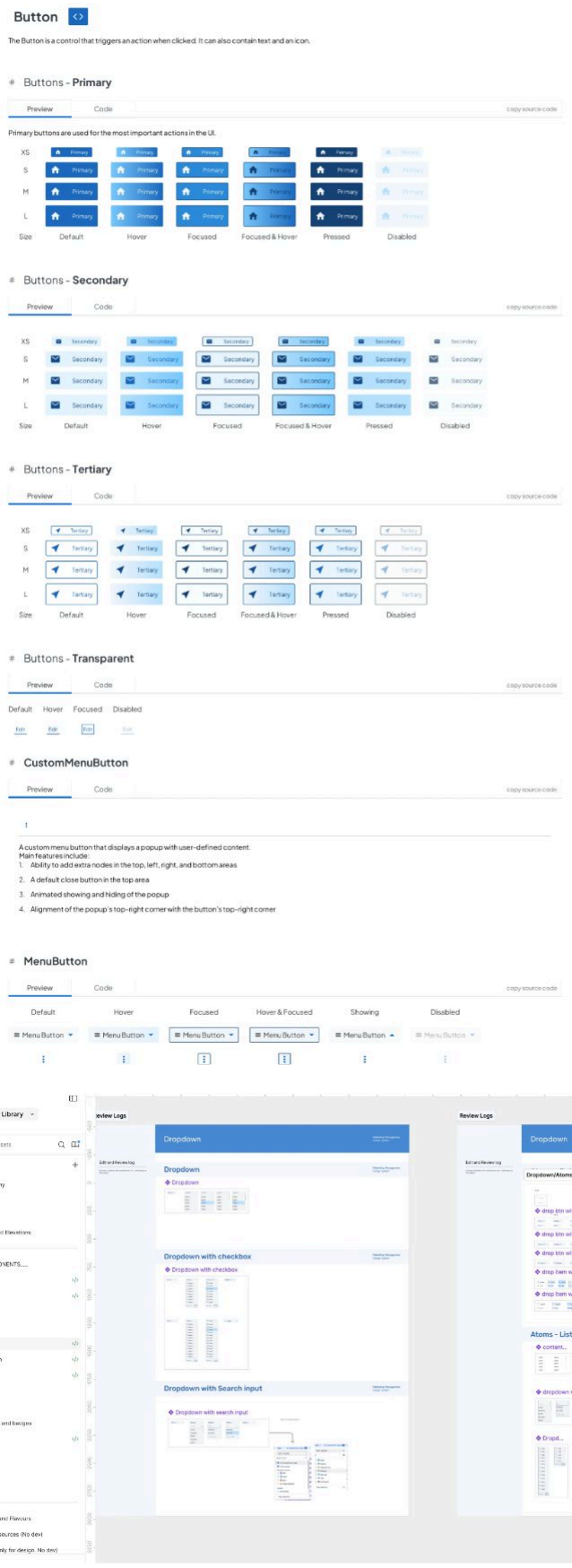
Using Modes

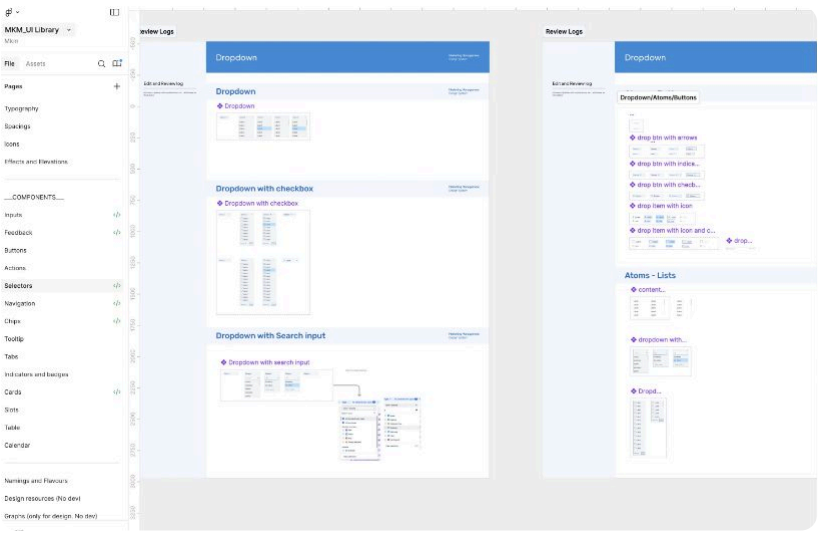
Modes are used as a tool across a small number of isolated collections, scoped to the component layer, for statuses, alert types and on/off, closer to a prototype approach.

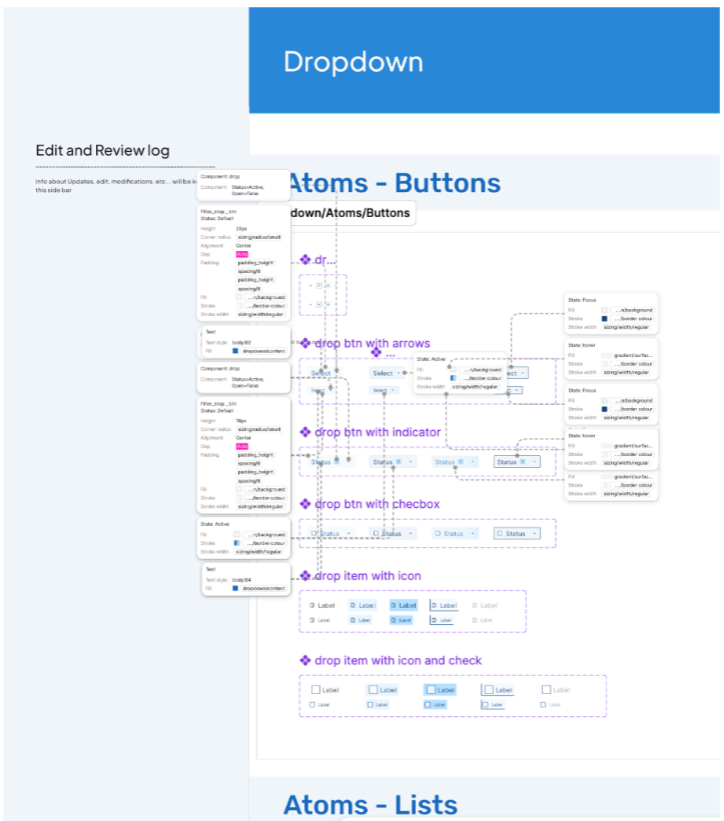
Tokens to Code Pipeline

A pair of custom GitHub Actions paired (later on) with a custom Figma plugin, shipping from Figma straight into the codebase as consumable web CSS and JavaFX-specific CSS, in an organised documentation-style format.

Zooming In: Shipping Tokens







An unusual stack requires a custom solution

This project stack was JavaFX, a language I was unfamiliar with and my research of design systems within it came back empty. It turned out it was also incompatible with every industry standard tool available at the time: Supernova, Storybook and Zeroheight were all ruled out, so the choice was made for me.

Tokens lived in Figma; the developer in Japan needed them as consumable JavaFX-specific CSS. Getting there took two attempts:

✗ First proposal: use Token Studio to get tokens as a JSON file. Denied; they asked for CSS files to be sent manually.

✓ Second proposal: have direct repo access with me pushing from my own machine. This was approved, though it meant manually copying and cleaning CSS output by hand for a stretch.

During this manual period, I was convinced a more efficient way existed out there, but knowing my limitations on the tech side, I reached out to a full-stack developer in the company to help me scope the problem. Together we scaffolded a solution and set it up. The eventual pipeline we ended up building used, initially, Token Studio as the "launchpad", pushing directly to the GitHub repo, where every push triggers two sets of GitHub Actions: one transforms it into a web CSS file (and overwrites the previous version), and a consecutive one that transforms the JSON into a JavaFX-specific CSS.

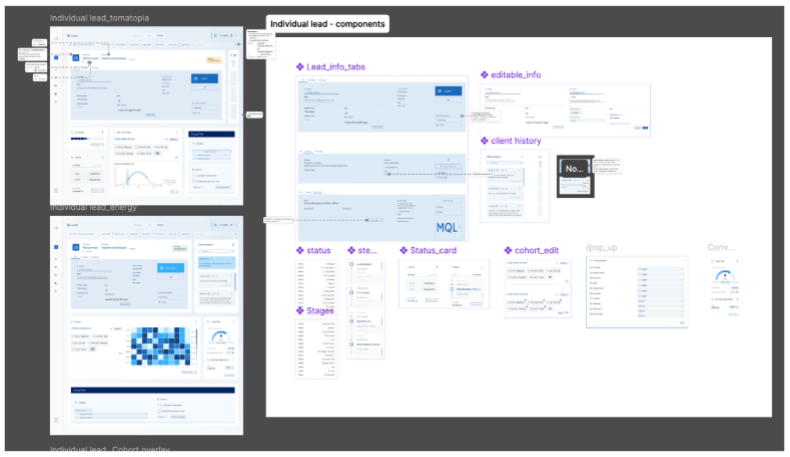
{for geeks, this is the reference point for those later transformations}

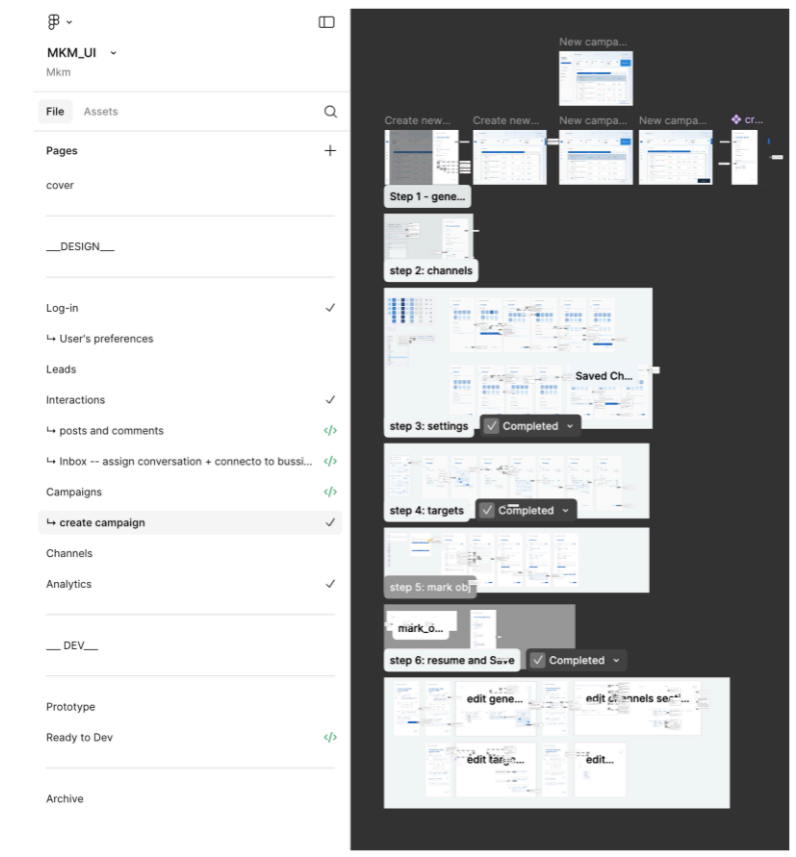
The Buy-in Moment

This particular pipeline was what, unexpectedly, ended up gaining the CEO's attention. He was independently dealing with the same async, cross-language friction with my frontend developer, on an unrelated issue, at the same time I was. In a catchup meeting, I walked him through my delivery tactics as well as this Figma-to-GitHub pipeline. He was impressed enough, and the reaction was immediate: "I want this running on all projects asap!!!!" triggering my official upgrade to Design Systems Lead and greenlight to audit and train the rest of the design team accordingly (of course, while still doing this project full time!)

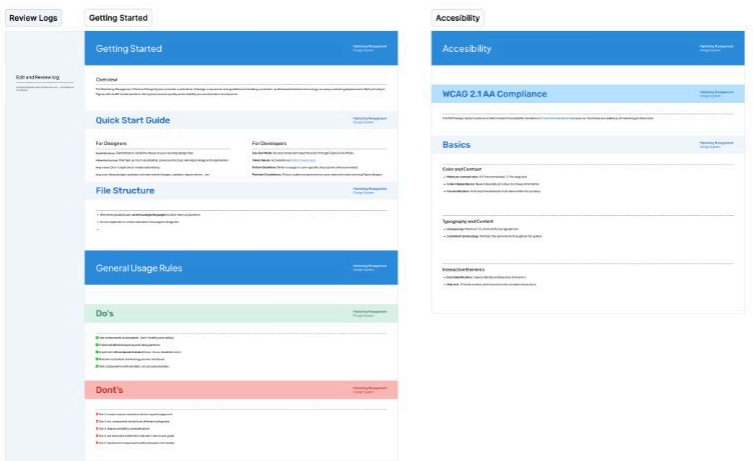
Now, by this point there were: 7 product lines running on 6 different dev stacks (React, Flutter, Swift, Nuxt, JavaFX, Framer). My approach to scaling was "education, not imposition": fill the knowledge gap first, then implement and follow through on rules and guidelines. That took the form of a five-part internal series, Design Bites: the 1.01 sessions, covering foundations, components and terminology, governance, design tokens, and naming systems, alongside hands-on workshops: brand and component self-audits, a shared design-principles exercise, an expert walkthrough on tokens, and first-iteration attempts at the company's own core and semantic tokens.

Reception was outstanding: changed behaviour across every team member but one, followed by individual conversations and requests from designers wanting to work out how the theory applied to their own product's daily reality.









Images of the design system in action, hand-off notifications and atomised deliveries and overall onboarding rules.

Quirks and Outcomes

~50% Faster Hand-off

Component-first delivery (build the pieces, annotate every state and edge case, assemble screens second) cut hand-off time by roughly half compared to other teams at the company, by informal comparison with other designers rather than controlled measurement. Set against the baseline it replaced: four or five review rounds per hand-off, sometimes corrections happening after a product had already gone live.

A Pipeline, Not a Hand-off Ritual

The side effect mattered as much as the fix: the same pipeline that turned token hand-off into a one-click export also unlocked white-label capability for an unrelated, earlier product, which is what first got the CEO's attention, independently of the design system pitch itself. Within roughly five to six months, that became a promotion to Design Systems Lead.

Documentation, Deferred on Purpose

No traditional component documentation existed for most of the build, a deliberate trade-off while the sole consumer of the system was its own author. It held up, but it also meant the system lived in one person's head for longer than was comfortable once the team started to grow, eventually scaling to 8 designers. That gap is part of what the later Design Bites series was built to close: five internal sessions and hands-on workshops, run once governance work began, rather than documentation written as the system was built. With earlier buy-in, component documentation would have started sooner, even lightweight, instead of waiting for a workshop series to catch up.

Learnings

The architectural call to add a fourth level rather than force those patterns into semantic, or rebuild the whole system to accommodate them, was the right one for the problem it solved; it stayed the right call even in hindsight. What I'd revisit is sequencing: with earlier buy-in, component documentation would have started sooner, even lightweight, instead of living in one person's head for the length of the build.

The governance model was still being tested when the company went bankrupt, before the scaling model reached full execution. The structure existed, and the reception from the team it was tested on was genuinely strong; the runway to prove it at full scale didn't.

Thanks for your time

www.

www.silviatravieso.com

@.

silvia.travieso.g@gmail.com



Silvia Travieso · 2026